

Homework3

February 11, 2019

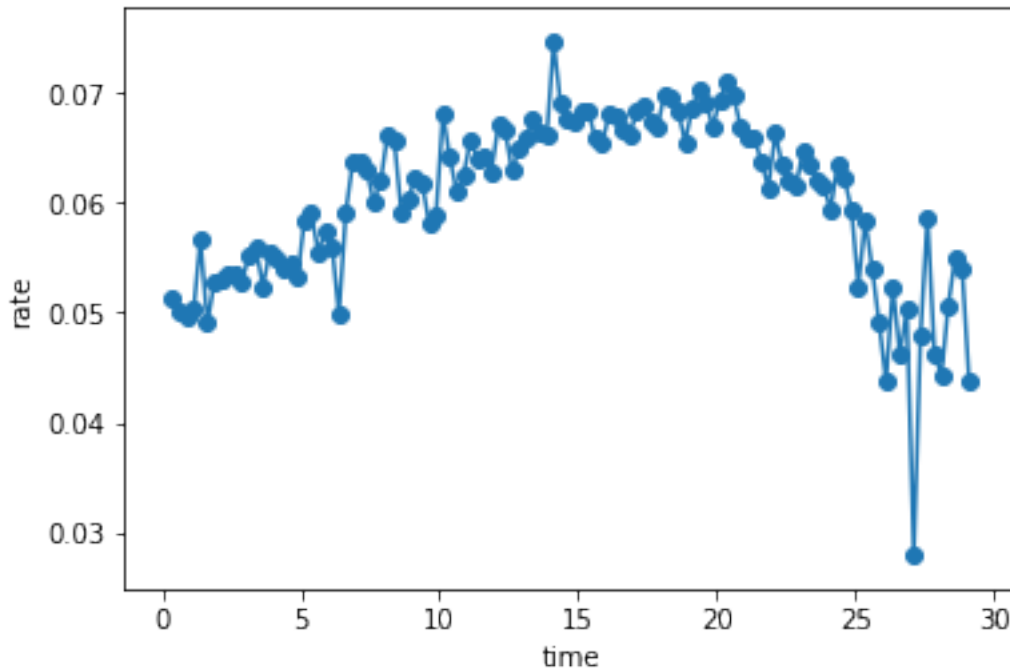
```
In [94]: import numpy as np
import sklearn as sk
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import mean_squared_error
from sklearn import metrics
import matplotlib
import matplotlib.pyplot as plt
import scipy.stats
import pandas as pd
from scipy.interpolate import InterpolatedUnivariateSpline
import pygam
from pygam import LinearGAM
from pygam import LogisticGAM

In [2]: matplotlib.rcParams['figure.figsize'] = (12,8)
```

1 Problem 1

(a)

```
In [110]: data = pd.read_csv('./forward_rates.csv')
plt.plot(data['time'], data['rate'])
plt.scatter(data['time'], data['rate'])
plt.xlabel('time')
plt.ylabel('rate')
plt.show()
```

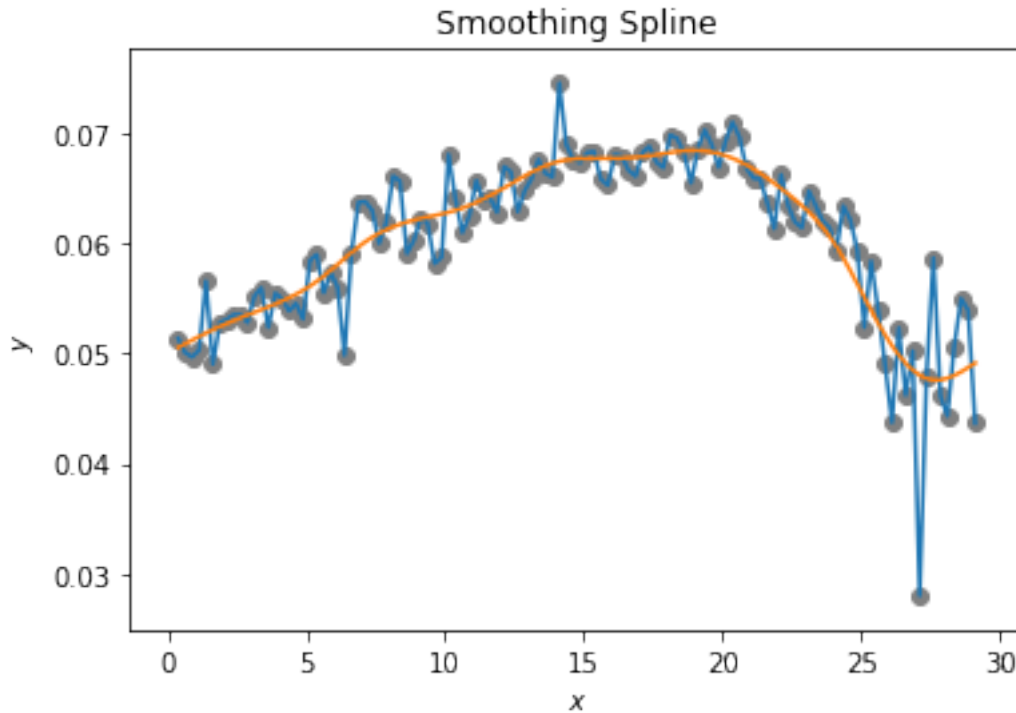


(b) As is shown below, the cross-validated lambda is: 0.48939, and I believe the fit by watching the graph.

```
In [47]: #Cross-validate for a reasonable lambda.
#Plot your fit on the same plot as the points. Can you manage to get a believable, smooth fit?
n = len(df['time'])
x = data['time'].values.reshape(n, 1)
y = data['rate'].values
fit = LinearGAM().gridsearch(x,y,lam = np.logspace(-5, 3, 30), keep_best=True)
print('Penalization term is: ', fit.lam) #is the penalization term that is multiplied by
#the second derivative in the overall objective function, in this case of a cubic
#plot
fig, ax = plt.subplots()
ax.plot(data['time'], data['rate'])
ax.plot(data['time'], fit.predict(data['time']))
ax.scatter(x, y, c='grey')
ax.set(xlabel = r"$x$",
      ylabel = r"$y$",
      title = r"Smoothing Spline");
plt.show()
```

100% (30 of 30) |#####| Elapsed Time: 0:00:00 Time: 0:00:00

Penalization term is: [[0.4893900918477489]]



- (c) We can write a K-knot spline as $f(x) = B_0 + B_1b_1(x) + \dots + B_{k+3}b_{k+3}(x)$ for every well behaved bases $b_1, \dots, b_{k+3}$, making computation fast and efficient. For this kind of optimization problem, the minimizer is always a cubic spline. Also, cubic splines are C2-continuous, thus easy for integration, and thus easy to get a closed form for forward rate.

2 Problem 2

(a)

```
In [58]: train = pd.read_csv('./bike_train.csv')
test = pd.read_csv('./bike_test.csv')
# train.head()
# rentals dayofyear, dayofweek, workday, weathertype, temp, humidity, windspeed.
x_train = train[['dayofyear', 'dayofweek', 'workday', 'weathertype', 'temp', 'humidity']]
y_train = train['rentals'].values
x_test = test[['dayofyear', 'dayofweek', 'workday', 'weathertype', 'temp', 'humidity']]
y_test = test['rentals'].values
model = LinearRegression()
model.fit(x_train, y_train)

y_pred = model.predict(x_test) * 1.6
MSE = mean_squared_error(y_pred, y_test)
print('MSE is: ', MSE)
```

MSE is: 1295159.3945082321

(b)

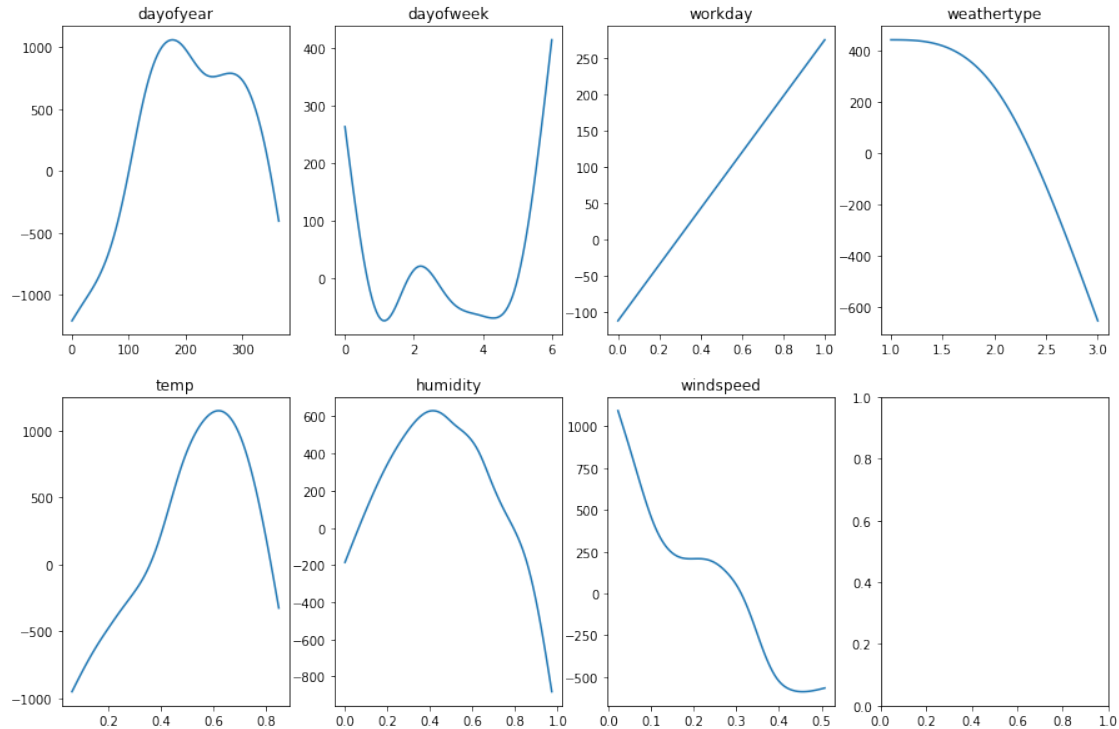
```
In [60]: add_fit = LinearGAM().gridsearch(x_train, y_train, lam = np.logspace(-5, 3, 30), keep.  
      y_hat = add_fit.predict(x_test) * 1.6  
      add_MSE = mean_squared_error(y_hat, y_test)  
      print("MSE: ", add_MSE)
```

100% (30 of 30) |#####| Elapsed Time: 0:00:02 Time: 0:00:02

MSE: 965230.5207073245

(c) Except "workday", almost all the other factors has nonlinear contributions. Keeping all the other factors unchanged, rental increases from spring to autumn, and decreases after that time period; rental is high during weekends and low during weekdays; rental decreases as the weather gets worse; rental increase as temperature gets higher but will drop until the temperature reaches some point; rental increase as humidity gets higher but will drop until the humidity reaches some point; rental decreases as wind blows harder. All the above interperation makes sense to me.

```
In [67]: fig, axs = plt.subplots(2, 4, figsize=(15,10))  
      # fig, axs = plt.subplots(2, 4)  
      # rentals dayofyear, dayofweek, workday, weathertype, temp, humidity, windspeed.  
      titles = ['dayofyear', 'dayofweek', 'workday', 'weathertype', 'temp', 'humidity', 'windspeed']  
  
      for i in range(7):  
          ax = axs[i//4, i%4]  
          Xfake = add_fit.generate_X_grid(i)  
          pdep, confi = add_fit.partial_dependence(term = i, X= Xfake, width=.95)  
  
          ax.plot(Xfake[:, i], pdep)  
          ax.set_title(titles[i])  
  
      plt.show()
```



(d)

```
In [68]: def real_loss(true, guess):
          return(np.mean(np.where(true>guess, 5*(true-guess), guess-true)))

          linear_reg_loss = real_loss(y_test, y_pred)
          add_loss = real_loss(y_test, y_hat)

          print('Loss of linear regression: ', linear_reg_loss)
          print('Loss of additive model: ', add_loss)
```

Loss of linear regression: 2412.8302850921377

Loss of additive model: 2094.8714512536117

(e) After doing the adjustment for both estimator, MSE of both model decrease by almost a half. Comparing these two MSE's, I will choose additive model since it captures nonlinearity and gives lower MSE than linear model.

```
In [69]: res_linear = y_test - y_pred
          res_add = y_test - y_hat

          q_linear = np.percentile(res_linear, 5/6*100)
          q_add = np.percentile(res_add, 5/6*100)
```

```

y_pred_new = y_pred + q_linear
y_hat_new = y_hat + q_add

linear_reg_loss_new = real_loss(y_test, y_pred_new)
add_loss_new = real_loss(y_test, y_hat_new)

print('Loss of linear regression: ', linear_reg_loss_new)
print('Loss of additive model: ', add_loss_new)

```

Loss of linear regression: 1429.4534002950518
 Loss of additive model: 1345.9830553584115

3 Problem 4

```

In [71]: market_data = pd.read_csv('./marketing.csv')
        Y = np.where(market_data['y'] == 'yes', 1, 0)
        X = market_data.drop(['y'], axis = 1)
        X = pd.get_dummies(X)
        xtrain, xtest, ytrain, ytest = train_test_split(X, Y, test_size=0.33, random_state=1)

```

(a)

```

In [72]: base_rate = sum(ytrain) / len(ytrain)
        print("The base rate is:", base_rate)

```

The base rate is: 0.11782377603908752

(b)

```

In [80]: logistic_model = LogisticRegression(C = 100000)
        log_model_fit = logistic_model.fit(xtrain, ytrain)
        y_pred = log_model_fit.predict(xtest)
        m_rate = np.mean(ytest != y_pred)
        print('Misclassification rate is:', m_rate)

```

/Users/rachel/anaconda3/lib/python3.7/site-packages/sklearn/linear_model/logistic.py:433: FutureWarning

Misclassification rate is: 0.11548257372654155

(c) Misclassification rate is lower compared to logistic classifier.

```

In [113]: # ytest
        silly_classifier = sum(ytest) / len(ytest)
        print('Misclassification rate is:', silly_classifier)

```

Misclassification rate is: 0.11528150134048257

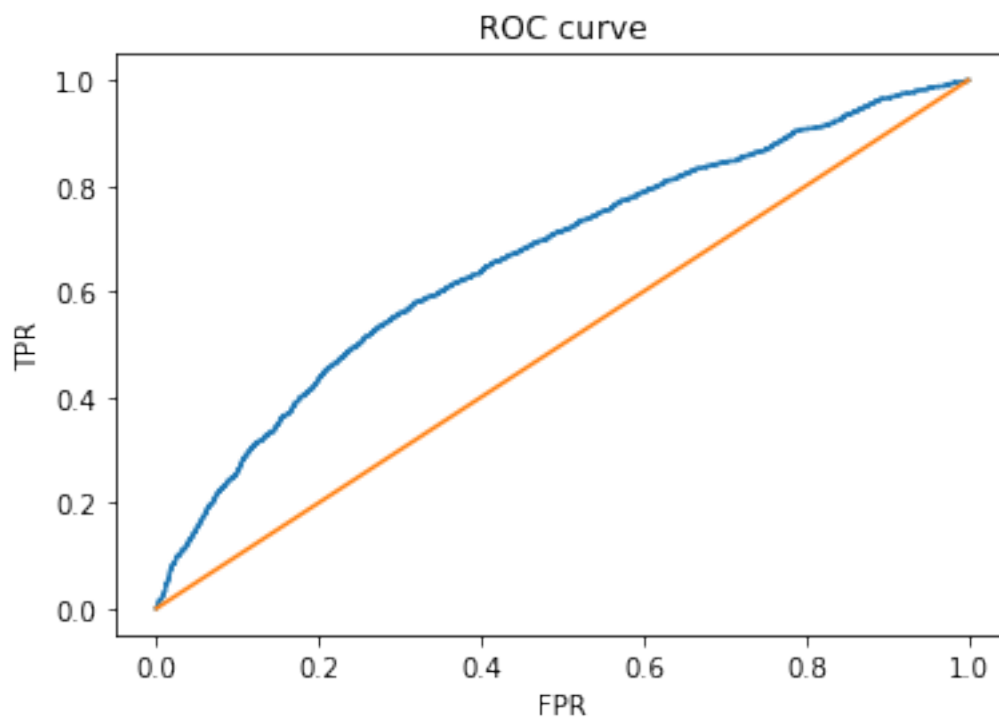
- (d) For the test set of 1000 clients, 27.9% of them actually says 'yes', but for a random set, I would expect this number to be 11.5%.

```
In [108]: n = 1000
          y_pred_dens = logistic_model.predict_proba(xtest)[: ,1]
          indices = np.flip(np.argsort(y_pred_dens))
          target = ytest[indices][0:1000]
          fraction = sum(target) / n
          print("Fratction is:", fraction)
```

Fratction is: 0.279

- (e)

```
In [92]: fpr, tpr, _ = metrics.roc_curve(ytest, yhat_prob)
          x = np.linspace(0, 1, 100)
          y = x
          plt.plot(fpr, tpr)
          plt.plot(x, y)
          plt.xlabel("FPR")
          plt.ylabel("TPR")
          plt.title("ROC curve")
          plt.show()
```



4 Problem 5

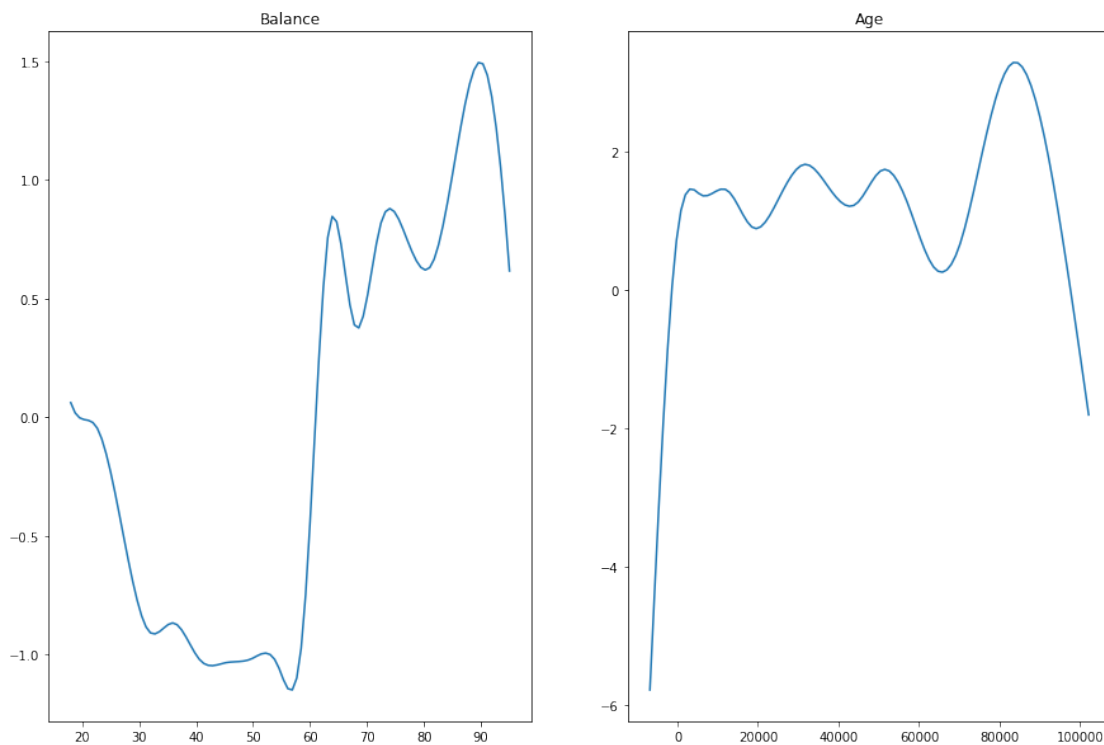
```
In [96]: logam = LogisticGAM().gridsearch(xtrain.values, ytrain, lam = np.logspace(-5, 3, 30),
100% (30 of 30) |#####| Elapsed Time: 0:06:11 Time: 0:06:11
```

(a)

```
In [98]: fig, axs = plt.subplots(1, 2, figsize=(15,10))
titles = ['Balance', 'Age']
for i, ax in enumerate(axs):
    Xfake = logam.generate_X_grid(i)
    pdep, confi = logam.partial_dependence(term = i, X= Xfake, width=.95)

    ax.plot(Xfake[:, i], pdep)
    ax.set_title(titles[i])

plt.show()
```



(b) Compared to 4(b) and 4(c), both misclassification rate and fraction of clients that will say 'yes' increases.

```
In [109]: log_y_pred = logam.predict(xtest)
print('The misclassification rate is :', np.sum(ytest != log_y_pred)/len(ytest))
```



```

logypred_dens = logam.predict_proba(xtest)
indices = np.flip(np.argsort(logypred_dens))
fraction = sum(ytest[indices][:1000] == 1) / 1000
print("Fraction is:", fraction)

```

The misclassification rate is : 0.11702412868632708

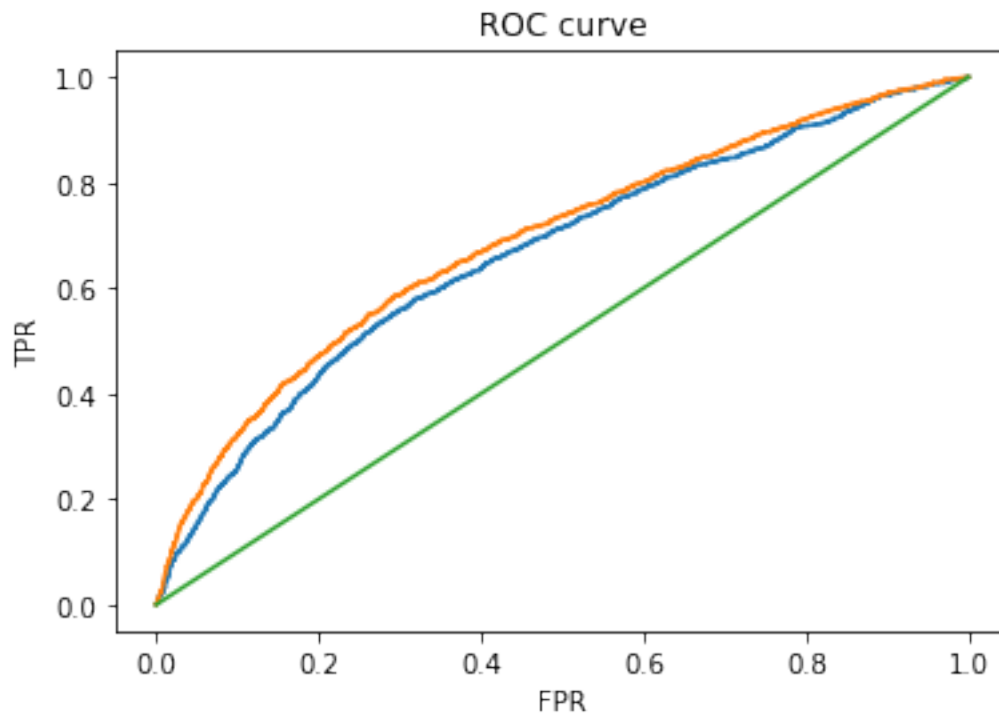
Fraction is: 0.346

(c)

```

In [112]: fprgam, tprgam, _ = metrics.roc_curve(ytest, logypred_dens)
x_range = np.linspace(0, 1, 100)
y_range = x_range
plt.plot(fpr, tpr)
plt.plot(fprgam, tprgam)
plt.plot(x_range, y_range)
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.title("ROC curve")
plt.show()

```



In []:

3. (a).

We have loss function.

$$L(f(x), y) = L_0(f(x)=1, y=0) + L_1(f(x)=0, y=1)$$

objective function: $\min \mathbb{E}(L(f(x), y))$

$$\begin{aligned}\Rightarrow \arg \min_{f(x)} \mathbb{E}(L(f(x), y)) &= \arg \min \mathbb{E}[L_0(f(x)=1, y=0) + L_1(f(x)=0, y=1)] \\ &= \arg \min \mathbb{E}[L_0(f(x)=1, y=0) + L_1(f(x)=0, y=1) | x=x] \\ &= \arg \min \mathbb{E}[\sum L_0(f(x)=1) \cdot P(y=0|x) \\ &\quad + \sum L_1(f(x)=0) \cdot P(y=1|x)]\end{aligned}$$

$$\Rightarrow f(x) = \begin{cases} 1 & , L_0 P(y=0|x=x) \leq L_1 P(y=1|x=x) \\ 0 & , \text{otherwise} \end{cases}$$

(b) $\because L_1 > L_0$

$\therefore$ The cost of guessing $y=1$ when it is actually $y=0$ is less than the cost of guessing $y=0$ when it is actually $y=1$, which means MORE penalty is given when we guess $y=0$ that actually it is 1.

$\therefore P(L_1) < \frac{1}{2} < P(L_0)$ to minimize total loss.

(c) By part (a), we know in this case $f(x) = 1$

$$\therefore L_0 P(y=1|x=x) \geq L_1 P(y=0|x=x)$$

$$\text{let } P(y=1|x=x) = b, \text{ then } P(y=0|x=x) = 1-b$$

$$\therefore \frac{b}{1-b} \geq \frac{L_1}{L_0}$$

Take $\log$ on both sides, $\log\left(\frac{b}{1-b}\right) \geq \log\left(\frac{L_1}{L_0}\right)$, where $\log\left(\frac{b}{1-b}\right) = x^T \hat{\beta}$

$\therefore$ The rule becomes:

$$\hat{f}(x) = \begin{cases} 1, & x^T \hat{\beta} \geq \log\left(\frac{L_1}{L_0}\right) \\ 0, & \text{otherwise.} \end{cases}$$

$\therefore$ when $L_0 > L_1$, the boundary is $\log\left(\frac{L_1}{L_0}\right)$.